```python
"""
validate.py — Input validation for patient risk and claim outcome models.

Usage:
    from validate import validate_payload
    ok, errors = validate_payload(data_dict, schema_dict)
"""

from typing import Any


def validate_payload(data: dict[str, Any], schema: dict) -> tuple[bool, list[str]]:
    """
    Validate a prediction payload against a feature schema.

    Parameters
    ----------
    data   : dict mapping feature name -> value (from the incoming request)
    schema : parsed JSON schema (as loaded from the feature schema JSON file)

    Returns
    -------
    (is_valid, errors)
        is_valid — True when no errors were found
        errors   — list of human-readable error strings (empty on success)
    """
    errors: list[str] = []

    for feature in schema.get("features", []):
        name = feature["name"]
        required = feature.get("required", False)
        constraints = feature.get("constraints", {})
        ftype = feature.get("type", "")

        # ── Check 1: Missing / null values ──────────────────────────────
        if name not in data or data[name] is None:
            if required:
                errors.append(f"'{name}' is required but missing or null.")
            continue  # skip further checks when value is absent

        value = data[name]

        # ── Check 2: Numeric range ──────────────────────────────────────
        if ftype in ("int8", "int32", "int64", "float32", "float64"):
            if not isinstance(value, (int, float)):
                errors.append(
                    f"'{name}' must be numeric (got {type(value).__name__})."
                )
            else:
                min_val = constraints.get("min_value")
                max_val = constraints.get("max_value")
                if min_val is not None and value < min_val:
                    errors.append(
                        f"'{name}' value {value} is below minimum allowed value {min_val}."
                    )
                if max_val is not None and value > max_val:
                    errors.append(
```

```
                    f"'{name}' value {value} exceeds maximum allowed value {max_val}."
                )

        # ── Check 3: Unseen categories ──────────────────────────────────
        if "allowed_values" in constraints:
            allowed = constraints["allowed_values"]
            if value not in allowed:
                errors.append(
                    f"'{name}' value {value!r} is not in the allowed set: {allowed}."
                )

    return (len(errors) == 0, errors)
```